

NumPy Tutorial 3: Array Operations — Add, Subtract, Multiply and More

This chapter covers mathematical operations in NumPy. After completing this tutorial, students will understand how to perform operations on entire arrays at once without using loops.

1. Introduction

In the previous tutorial, we learned how to create different types of arrays. In this tutorial, we will learn how to perform mathematical operations on those arrays.

The most powerful feature of NumPy is that we can perform operations on entire arrays at once. This is called **vectorization** and it makes NumPy extremely fast.

2. What is Vectorization

In normal Python, if we want to add 10 to every number in a list, we have to use a loop.

```
# Python list way
numbers = [1, 2, 3, 4, 5]
result = []
for n in numbers:
    result.append(n + 10)
print(result)
```

With NumPy, we do the same thing in one line — no loop needed.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
result = arr + 10
print(result)
```

Output:

```
[11 12 13 14 15]
```

This is vectorization — NumPy applies the operation to every element automatically. This is much faster than using loops.

3. Basic Arithmetic Operations

NumPy supports all basic arithmetic operations directly on arrays.

Addition

```
import numpy as np

arr = np.array([10, 20, 30, 40])
print(arr + 5)
```

Output:

```
[15 25 35 45]
```

Subtraction

```
print(arr - 5)
```

Output:

```
[5 15 25 35]
```

Multiplication

```
print(arr * 2)
```

Output:

```
[20 40 60 80]
```

Division

```
print(arr / 2)
```

Output:

```
[ 5. 10. 15. 20.]
```

Power (Exponent)

```
print(arr ** 2)
```

Output:

```
[ 100  400  900 1600]
```

This raises every element to the power of 2.

4. Operations Between Two Arrays

We can also perform operations between two arrays of the same size.

```
import numpy as np

arr1 = np.array([10, 20, 30, 40])
arr2 = np.array([1, 2, 3, 4])

print(arr1 + arr2)
print(arr1 - arr2)
print(arr1 * arr2)
print(arr1 / arr2)
```

Output:

```
[11 22 33 44]
[ 9 18 27 36]
[ 10  40  90 160]
[10. 10. 10. 10.]
```

NumPy performs the operation **element by element** — first element with first, second with second, and so on.

5. Mathematical Functions

NumPy provides many built in mathematical functions.

Square Root

```
arr = np.array([4, 9, 16, 25])
print(np.sqrt(arr))
```

Output:

```
[2.  3.  4.  5.]
```

Absolute Value

```
arr = np.array([-5, -3, 0, 3, 5])
print(np.abs(arr))
```

Output:

```
[5 3 0 3 5]
```

Floor and Ceil

```
arr = np.array([1.2, 2.7, 3.5, 4.9])
print(np.floor(arr))
print(np.ceil(arr))
```

Output:

```
[1.  2.  3.  4.]
[2.  3.  4.  5.]
```

Floor rounds down. Ceil rounds up.

Logarithm

```
arr = np.array([1, 10, 100, 1000])  
print(np.log10(arr))
```

Output:

```
[0.  1.  2.  3.]
```

6. Aggregate Functions

Aggregate functions calculate a single value from the entire array.

```
import numpy as np  
  
arr = np.array([10, 20, 30, 40, 50])  
  
print(np.sum(arr))  
print(np.min(arr))  
print(np.max(arr))  
print(np.mean(arr))  
print(np.median(arr))  
print(np.std(arr))
```

Output:

```
150  
10  
50  
30.0  
30.0  
14.142135623730951
```

Function	Description
np.sum	Total of all elements
np.min	Smallest value
np.max	Largest value
np.mean	Average value

Function	Description
np.median	Middle value
np.std	Standard deviation

7. Operations on 2D Arrays

For 2D arrays, we can apply operations along rows or columns using the **axis** parameter.

- **axis=0** — operation along columns (top to bottom)
- **axis=1** — operation along rows (left to right)

```
import numpy as np

arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])

print(np.sum(arr, axis=0))
print(np.sum(arr, axis=1))
```

Output:

```
[12 15 18]
[ 6 15 24]
```

In this example:

- axis=0 adds each column together — $1+4+7=12$, $2+5+8=15$, $3+6+9=18$.
- axis=1 adds each row together — $1+2+3=6$, $4+5+6=15$, $7+8+9=24$.

8. Comparison Operations

We can compare arrays with a value — the result is a boolean array.

```
arr = np.array([10, 20, 30, 40, 50])

print(arr > 25)
print(arr == 30)
```

Output:

```
[False False  True  True  True]
[False False  True False False]
```

This is very useful for filtering data — we will use this in the Boolean Indexing tutorial.

9. Conclusion of Tutorial 3

In this tutorial, students have learned:

1. What vectorization is and why it makes NumPy fast.
2. How to perform basic arithmetic operations on arrays.
3. How to perform operations between two arrays.
4. How to use mathematical functions like `sqrt`, `abs`, `floor`, `ceil`.
5. How to use aggregate functions like `sum`, `mean`, `max`, `min`, `std`.
6. How to use `axis` parameter for 2D array operations.
7. How comparison operations work on arrays.

In the next tutorial, we will learn how to access specific elements from arrays using indexing and slicing.

This completes NumPy Tutorial 3.